

High-Level Design (HLD)

RAG-Based Customer Support Assistant with LangGraph & HITL

1. System Overview

This system is designed to automate customer support by retrieving precise technical information from a PDF knowledge base. Unlike standard chatbots, this system uses **Retrieval-Augmented Generation (RAG)** to prevent hallucinations and a **Graph-based workflow** to handle complex decision-making and human escalation.

2. Architecture Design

The architecture is split into two primary pipelines: the **Ingestion Pipeline** (Offline) and the **RAG Inference Pipeline** (Online).

[Architecture Diagram: PDF -> Chunking -> Embeddings -> Vector Store -> LLM -> LangGraph Logic]

2.1 Ingestion Pipeline

Processes raw PDF documents into a searchable mathematical format.

- **Document Loader:** Extracts text from PDF files.
- **Text Splitter:** Breaks text into 1000-character chunks with 200-character overlap to preserve context.
- **Embedding Engine:** Converts text into 384-dimensional vectors.
- **Vector Store:** Stores vectors in ChromaDB for high-speed similarity search.

2.2 Inference & Graph Logic

Governs how the system thinks and responds to user queries.

LangGraph Orchestrator: Manages the state and controls the flow between retrieval, generation, and escalation nodes.

3. Detailed Component Description

3.1 Vector Database ChromaDB

Acts as the long-term memory of the assistant. It performs a "Nearest Neighbor" search to find the most relevant document sections based on the user's question.

3.2 Large Language Model Groq / Llama-3

The reasoning engine. It takes the retrieved document snippets and synthesizes a natural language answer. It is restricted to only use the provided context to ensure accuracy.

3.3 Human-in-the-Loop (HITL)

A critical safety layer. If the system's "confidence" (context relevance) is low, the workflow pauses and routes the query to a human agent, ensuring the customer never receives a wrong or "I don't know" response without oversight.

4. Logic Flow (LangGraph)

The system operates as a State Machine:

1. **START:** User submits a query.
2. **RETRIEVE:** Search ChromaDB for relevant data.
3. **GENERATE:** Attempt to form an answer.
4. **CONDITION:** Is the context sufficient?
 - Yes: Return Answer -> **END**.
 - No: Route to **ESCALATE** node.
5. **ESCALATE:** Human provides manual input -> **END**.